On the Prevalence and Usage of Commit Signing on GitHub

A Longitudinal and Cross-Domain Study

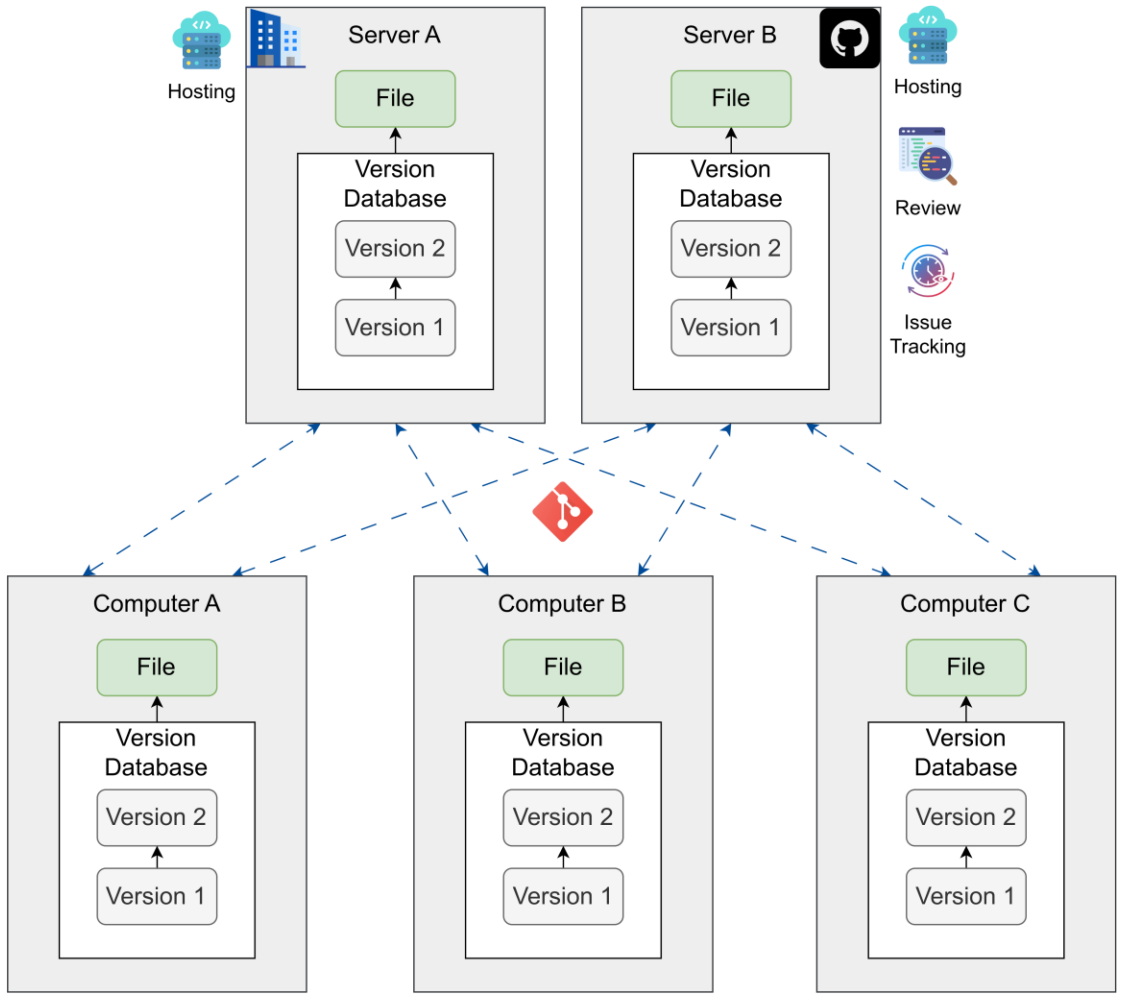
Anupam Sharma, Sreyashi Karmakar, Gayatri Priyadarsini Kancherla, Abhishek Bichhawat

Computer Science & Engineering
Indian Institute of Technology Gandhinagar

IIT Gandhinagar

29th Edition EASE 2025
Evaluation and Assessment in Software Engineering
17-20 JUNE | ISTANBUL.TÜRKİYE

Background: Git and GitHub



Git is a Distributed Version Control Systems. where clients fully mirror the repository, including its full history. If a server fails while the clients were collaborating via that server, any of the client repositories can be uploaded to the server to recover it. **GitHub** is a git-based repository hosting service provider which also provides services like, code review, issue tracking, pull request, etc. *Source: https://git-scm.com/book/en/v2*

Background: Commit Spoofing

Spoofer: dummy46

Victim: dummy400

1

Attacker clones victim's public repo and get username and email from git logs

```
efgh on  main
❯ git log --pretty=fuller
commit 9565b15da60581ee7a525f998f01c1d3ab34bd57 (HEAD → main, origi
Author:     dummy400 <182397874+dummy400@users.noreply.github.com>
AuthorDate: Sun Nov 3 17:50:54 2024 +0530
Commit:     GitHub <noreply@github.com>
CommitDate: Sun Nov 3 17:50:54 2024 +0530

    Create q.txt
```

Background: Commit Spoofing

Spoofer: dummy46

Victim: dummy400

1

Attacker clones victim's public repo and get username and email from git logs

```
efgh on  main
❯ git log --pretty=fuller
commit 9565b15da60581ee7a525f998f01c1d3ab34bd57 (HEAD → main, origi
Author:     dummy400 <182397874+dummy400@users.noreply.github.com>
AuthorDate: Sun Nov 3 17:50:54 2024 +0530
Commit:     GitHub <noreply@github.com>
CommitDate: Sun Nov 3 17:50:54 2024 +0530

    Create q.txt
```

2

Attacker configures local git config with user's identity

```
efgh on  main
❯ git config --local user.name "dummy400"

efgh on  main
❯ git config --local user.email \
· 182397874+dummy400@users.noreply.github.com

efgh on  main
❯ git config --local --get-regexp "user"
user.name dummy400
user.email 182397874+dummy400@users.noreply.github.com
```

Background: Commit Spoofing

Spoofer: dummy46

Victim: dummy400

1. Attacker clones victim's public repo and get username and email from git logs

```
efgh on  main
❯ git log --pretty=fuller
commit 9565b15da60581ee7a525f998f01c1d3ab34bd57 (HEAD → main, origi
Author:     dummy400 <182397874+dummy400@users.noreply.github.com>
AuthorDate: Sun Nov 3 17:50:54 2024 +0530
Commit:     GitHub <noreply@github.com>
CommitDate: Sun Nov 3 17:50:54 2024 +0530

    Create q.txt
```

2. Attacker configures local git config with user's identity

```
efgh on  main
❯ git config --local user.name "dummy400"

efgh on  main
❯ git config --local user.email \
· 182397874+dummy400@users.noreply.github.com

efgh on  main
❯ git config --local --get-regexp "user"
user.name dummy400
user.email 182397874+dummy400@users.noreply.github.com
```

3. Attackers commits & push a malicious commit making victim as committer/author which is unknown to the victim

Background: Commit Spoofing

Spoofer: dummy46

Victim: dummy400

1 Attacker clones victim's public repo and get username and email from git logs

```
efgh on  main
❯ git log --pretty=fuller
commit 9565b15da60581ee7a525f998f01c1d3ab34bd57 (HEAD → main, origi
Author:     dummy400 <182397874+dummy400@users.noreply.github.com>
AuthorDate: Sun Nov 3 17:50:54 2024 +0530
Commit:     GitHub <noreply@github.com>
CommitDate: Sun Nov 3 17:50:54 2024 +0530

    Create q.txt
```

2 Attacker configures local git config with user's identity

```
efgh on  main
❯ git config --local user.name "dummy400"

efgh on  main
❯ git config --local user.email \
· 182397874+dummy400@users.noreply.github.com

efgh on  main
❯ git config --local --get-regexp "user"
user.name dummy400
user.email 182397874+dummy400@users.noreply.github.com
```

3 Attackers commits & push a malicious commit making victim as committer/author which is unknown to the victim

4 GitHub showing victim as the committer which was actually committed by the attacker

5 Victim remains unaware about the commit. This activity is not shown in victim's GitHub feed, nor the repository belongs to the victim

Commit spoofing: The attacker (dummy46) gets the identity information from the victims's (dummy400) public repository logs and uses it in the Git config to push malicious commits on victim's behalf. GitHub associates the commit to the victim although the commit was made via the attacker.

.

Background: Commit Spoofing

Spoofer: dummy46 Victim: dummy400

1 Attacker clones victim's public repo and get username and email from git logs

```
efgh on  main
❯ git log --pretty=fuller
commit 9565b15da60581ee7a525f998f01c1d3ab34bd57 (HEAD → main, origi
Author:     dummy400 <182397874+dummy400@users.noreply.github.com>
AuthorDate: Sun Nov 3 17:50:54 2024 +0530
Commit:     GitHub <noreply@github.com>
CommitDate: Sun Nov 3 17:50:54 2024 +0530

    Create q.txt
```

2 Attacker configures local git config with user's identity

```
efgh on  main
❯ git config --local user.name "dummy400"

efgh on  main
❯ git config --local user.email \
· 182397874+dummy400@users.noreply.github.com

efgh on  main
❯ git config --local --get-regexp "user"
user.name dummy400
user.email 182397874+dummy400@users.noreply.github.com
```

3 Attackers commits & push a malicious commit making victim as committer/author which is unknown to the victim

4 GitHub showing victim as the committer which was actually committed by the attacker

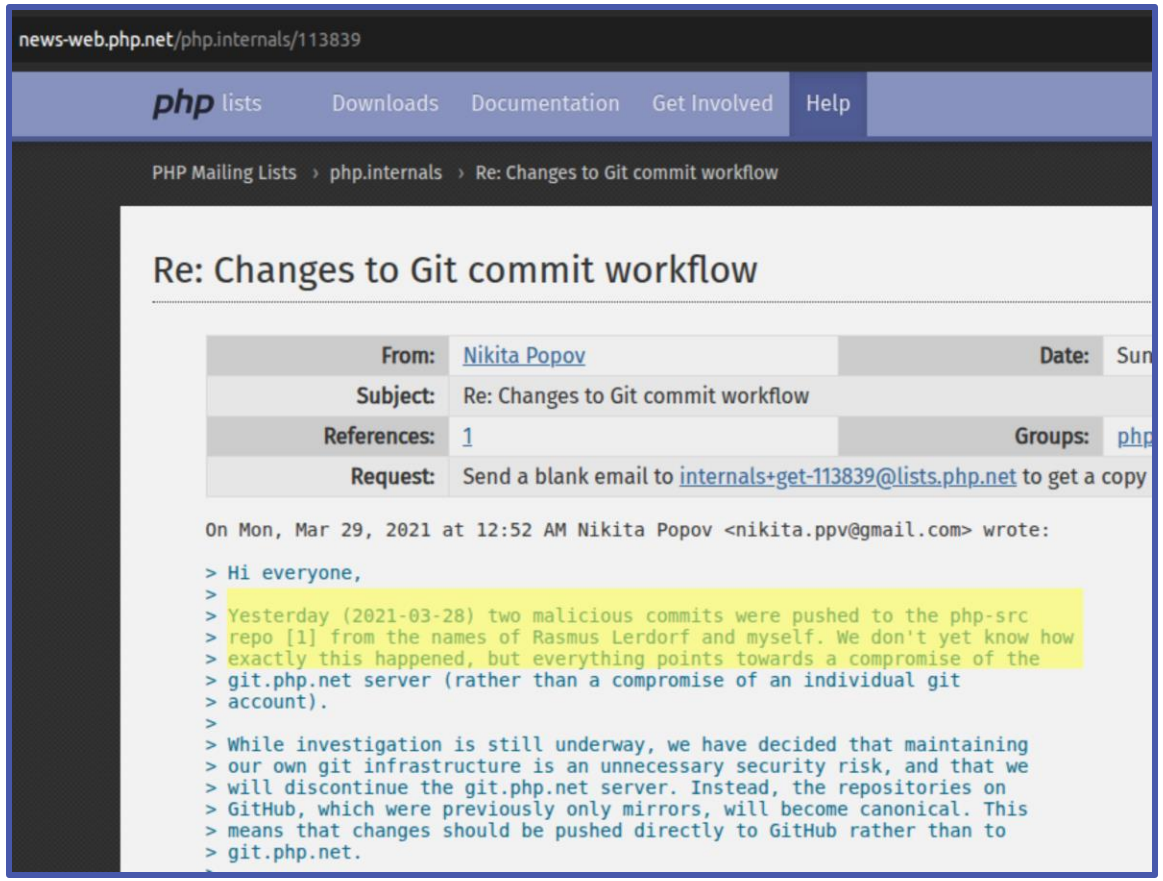
5 Victim remains unaware about the commit. This activity is not shown in victim's GitHub feed, nor the repository belongs to the victim

Commit spoofing: The attacker (dummy46) gets the identity information from the victims's (dummy400) public repository logs and uses it in the Git config to push malicious commits on victim's behalf. GitHub associates the commit to the victim although the commit was made via the attacker.

- Why? -> GitHub associates commit with the account via the email in commit metadata.
- **Consequence:** Anything can be published in the name of some other user.

Consequence: Malicious Activity via Impersonation

- Introduce malicious code in the name of other developer.



Malicious commits pushed to the php-src repository using the names of legitimate users [1].

[1] Nikita Popov. 2021. Changes to Git commit workflow. https://news-web.php.net/php.internals/113839

r/ProgrammerHumor • 4 mo. ago
Progractor

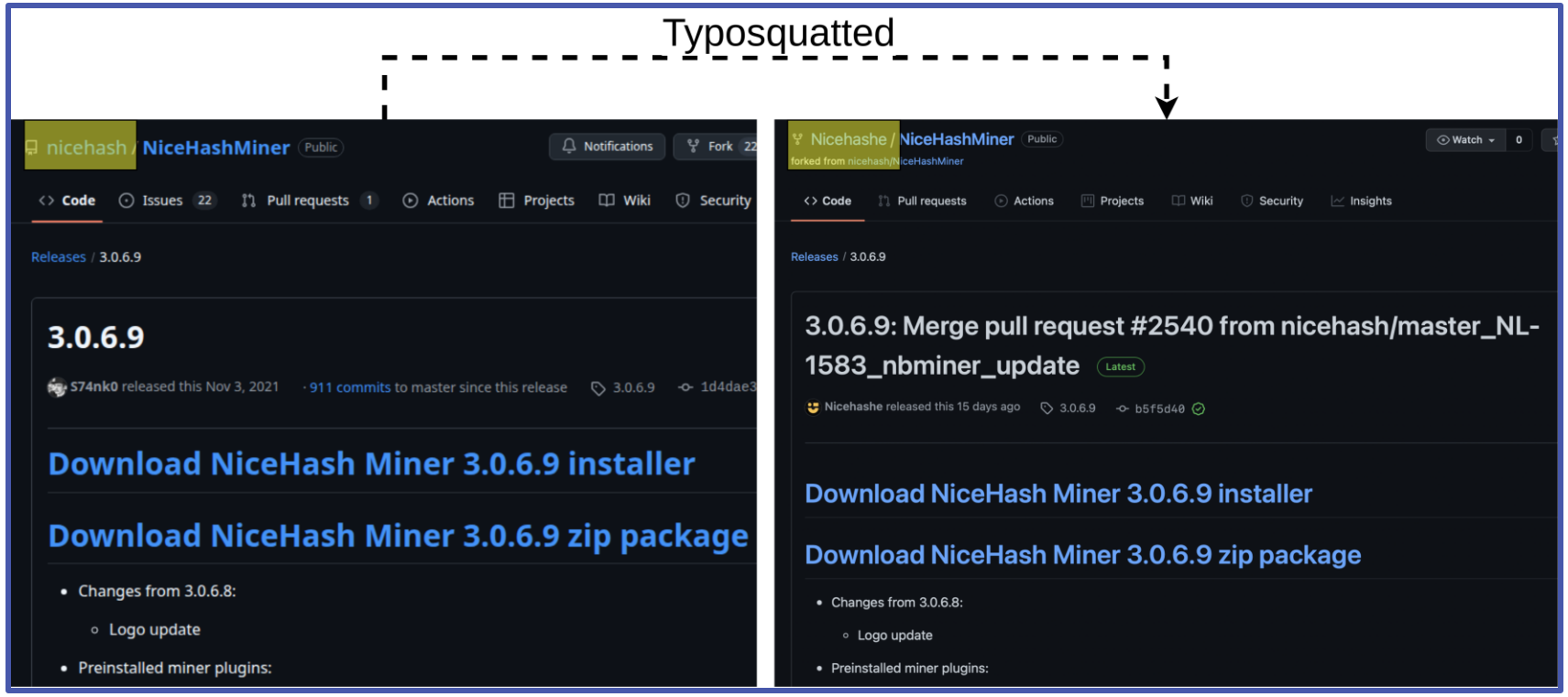
reddit

gitConfigImpersonation

Meme

Consequence: Typosquatting

- Fake repo (clone/fork) with slight change in name with malicious content.



Forked repository named "Nicehashe", typosquatting a legit repository "nicehash" with a malicious release. This was detected by Fork Sentry [2] and was later taken down by GitHub Trust & Safety. Such repositories may look legitimate via spoofed commits. More over, the "Forked from" label can be removed by cloning and creating a new repository.

[2] Alan Cao and Brendan Dolan-Gavitt. 2022. What the fork? finding and analyzing malware in github forks. In Proc. of NDSS, Vol. 22.

Solution?

- Can we prevent commit spoofing?
  - No. Because Git allows any identity information in git config.
- Alternative?
  - Commit Signing: Inform that commit is from the original owner by signing the commit.
    - Generate a GPG/SSH key pairs.
    - Sign the commit via the private key.
    - Share the public key with the verifier (For example: GitHub).
    - GitHub assigns badges to correctly signed commits.
  - Vigilant mode: A feature of GitHub to flag unsigned commit.

Commit Signing and Vigilant Mode

Correctly signed commit. GitHub assigns verified badge to correctly signed commits irrespective of the vigilant mode.

Commit Signing and Vigilant Mode

Correctly signed commit. GitHub assigns verified badge to correctly signed commits irrespective of the vigilant mode.

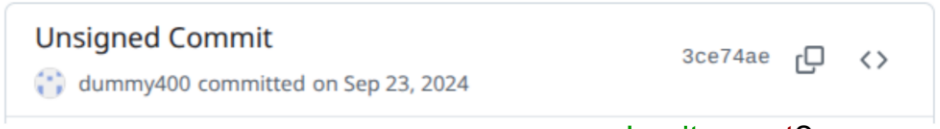
Vigilant mode off

Vigilant mode On

Commit Signing and Vigilant Mode

Correctly signed commit. GitHub assigns verified badge to correctly signed commits irrespective of the vigilant mode.

Vigilant mode off



Legit or not?
We don't know.

Vigilant mode On

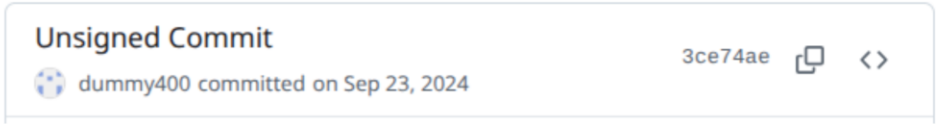
Seems like not legit.
Whatif it was legit and user was unaware of commit signing?

Unsigned commit. GitHub assigns no badge if vigilant mode is off. Unverified badge is assigned when vigilant mode is on, flagging a possibly spoofed commits. But it might be a legit commit which remained unsigned by mistake, It still gets an unverified badge. Still, we don't know if it is legit or not.

Commit Signing and Vigilant Mode

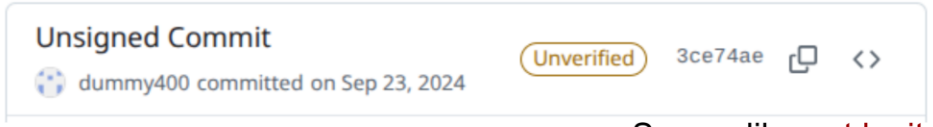
Correctly signed commit. GitHub assigns verified badge to correctly signed commits irrespective of the vigilant mode.

Vigilant mode off



Legit or not?
We don't know.

Vigilant mode On

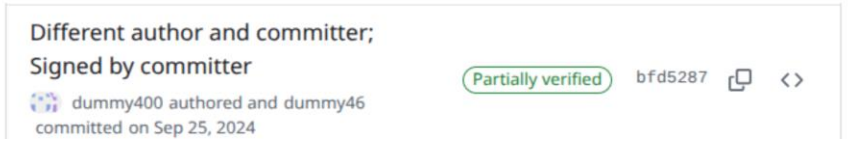


Seems like not legit.
Whatif it was legit and user was unaware of commit signing?

Unsigned commit. GitHub assigns no badge if vigilant mode is off. Unverified badge is assigned when vigilant mode is on, flagging a possibly spoofed commits. But it might be a legit commit which remained unsigned by mistake, It still gets an unverified badge. Still, we don't know if it is legit or not.

Vigilant mode off

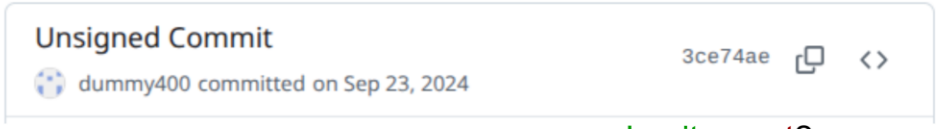
Vigilant mode On



Commit Signing and Vigilant Mode

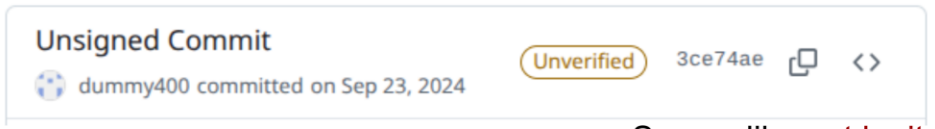
Correctly signed commit. GitHub assigns verified badge to correctly signed commits irrespective of the vigilant mode.

Vigilant mode off



Legit or not?
We don't know.

Vigilant mode On



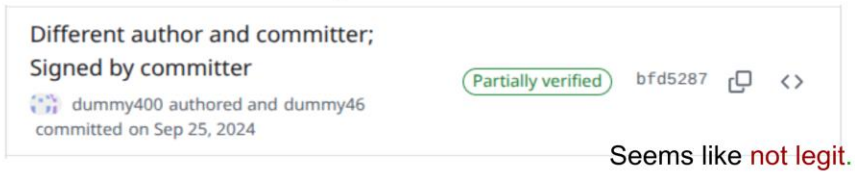
Seems like not legit.
Whatif it was legit and user was unaware of commit signing?

Unsigned commit. GitHub assigns no badge if vigilant mode is off. Unverified badge is assigned when vigilant mode is on, flagging a possibly spoofed commits. But it might be a legit commit which remained unsigned by mistake, It still gets an unverified badge. Still, we don't know if it is legit or not.

Vigilant mode off

Seems like Legit
What if author's identity was used via git commit --author option and the actual owner is unaware about this? Then it is not legit. Still a verified badge.

Vigilant mode On



Seems like not legit.
Whatif it was legit and user was unaware of commit signing?

Partially signed commit. When the author and committer is different, only committer can sign the commits.

Commit Signing and Vigilant Mode

- The interpretation of the badges could be vague based on context.
- Success of this process depends highly on awareness of:
  - Commit spoofing
  - Commit signing
- But are users aware of such concepts?
- Do they use commit signing?
- Which communities are more aware of such issues?

Measurment Study

- Analysed 56 months of commit metadata of top repositories from 4 different domains.

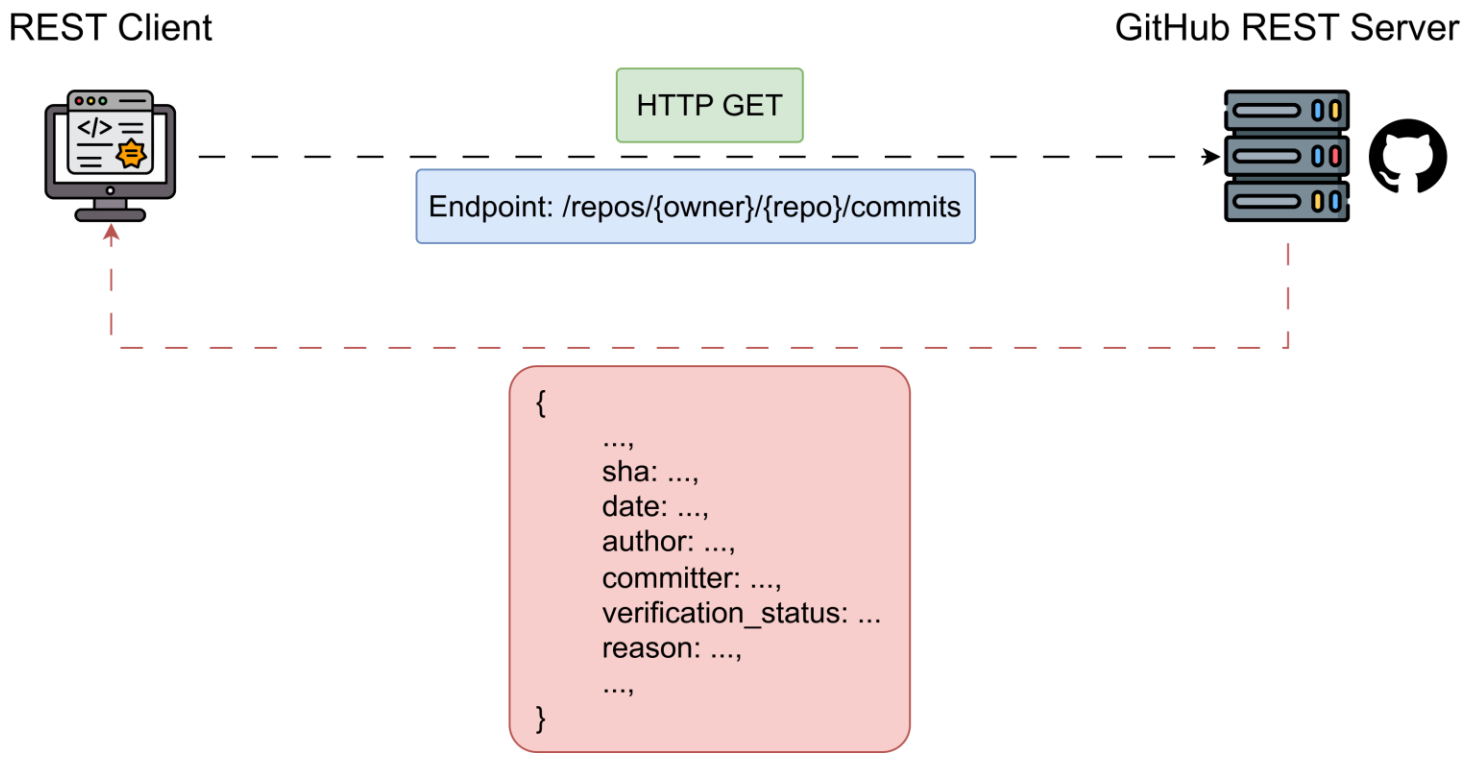
Sl. No.	Machine Learning	Database	Security	Web dev
1	tensorflow/tensorflow	redis/redis	fatedier/frp	facebook/react
2	AUTOMATIC1111/stable-diffusion-webui	ClickHouse/ClickHouse	NationalSecurityAgency/ghidra	vercel/next.js
3	huggingface/transformers	pingcap/tidb	gorhill/uBlock	facebook/react-native
4	pytorch/pytorch	cockroachdb/cockroach	mitmproxy/mitmproxy	nodejs/node
5	tensorflow/models	influxdata/influxdb	rapid7/metasploit-framework	mrdoob/three.js
6	keras-team/keras	facebook/rocksdb	gchq/CyberChef	denoland/deno
7	scikit-learn/scikit-learn	surrealdb/surrealdb	schollz/croc	angular/angular
8	geekan/MetaGPT	mongodb/mongo	openssl/openssl	mui/material-ui
9	hiyouga/LLaMA-Factory	duckdb/duckdb	AdguardTeam/AdGuardHome	ant-design/ant-design
10	microsoft/autogen	taosdata/TDengine	brave/brave-browser	puppeteer/puppeteer
11	microsoft/DeepSpeed	pubkey/rxdb	cure53/DOMPurify	tauri-apps/tauri
12	openai/gym	timescale/timescaledb	zaproxy/zaproxy	storybookjs/storybook
13	ray-project/ray	rqlite/rqlite	cryptomator/cryptomator	sveltejs/svelte
14	hankcs/HanLP	tikv/tikv	SoftEtherVPN/SoftEtherVPN	gin-gonic/gin
15	huggingface/pytorch-image-models	scylladb/scylladb	OpenVPN/openvpn	gohugoio/hugo

Top 15 repositories related to tools or libraries having at least 1K commits were considered for the study based on their popularity in Gitstar Ranking [3] for each of the 4 categories. This list excluded repositories having less than 20% of total commits during our extraction interval (28 April 2020 to 31 December 2024)

[3] Gitstar Ranking - https://gitstar-ranking.com/

Measurment Study

- Fetched 56 months of commit metadata of top repositories from 4 different domains.



REST Client. One of the component of the tool is a REST client that use the GitHub's REST API service to fetch the commit metadata from the repositories in GitHub.

Measurment Study

- Analysis of the commit metadata.

```
❯ python app.py analyze --task-name aiml1
WARNING: No bots provided. Will not exclude bots
Repo: tensorflow/tensorflow
Total commits: 89277
```

1

```
 Top committers (Likely bots):

COMMITTER
gardener@tensorflow.org                77270
noreply@github.com                      1863
yong.tang.github@outlook.com             291
vikoth18@in.ibm.com                      278
advaitjain@users.noreply.github.com      270
Name: count, dtype: int64
```

2

```
Proportion of verified commits:

+------------+---------+--------------------------+
|            | Overall | Excluding bots and GH web |
+------------+---------+--------------------------+
|  Verified  |  2206   |           343            |
| Unverified |  87071  |          87071           |
+------------+---------+--------------------------+

Proportion of specific reasons for unverified commits:

+------------------+---------+--------------------------+
|                  | Overall | Excluding bots and GH web |
+------------------+---------+--------------------------+
|     unsigned     |  86763  |          86763           |
|      valid       |  2206   |           343            |
|   unknown_key    |   24    |            24            |
|     no_user      |    6    |             6            |
| unverified_email |   278   |           278            |
+------------------+---------+--------------------------+
```

3

```
Yearwise distribution of verified commits excluding bots and GH web:

+-------+----------+-------+
|       | Verified | Total |
+-------+----------+-------+
| 20-21 |    86    | 24268 |
| 21-22 |   229    | 18481 |
| 22-23 |    10    | 18364 |
| 23-24 |    15    | 15966 |
| 24-25 |    3     | 10335 |
+-------+----------+-------+
```

4

```
Unique committers excluding bots and GH web:

Total unique committers: (793,)
Total unique committers with valid commits: (32,)
```

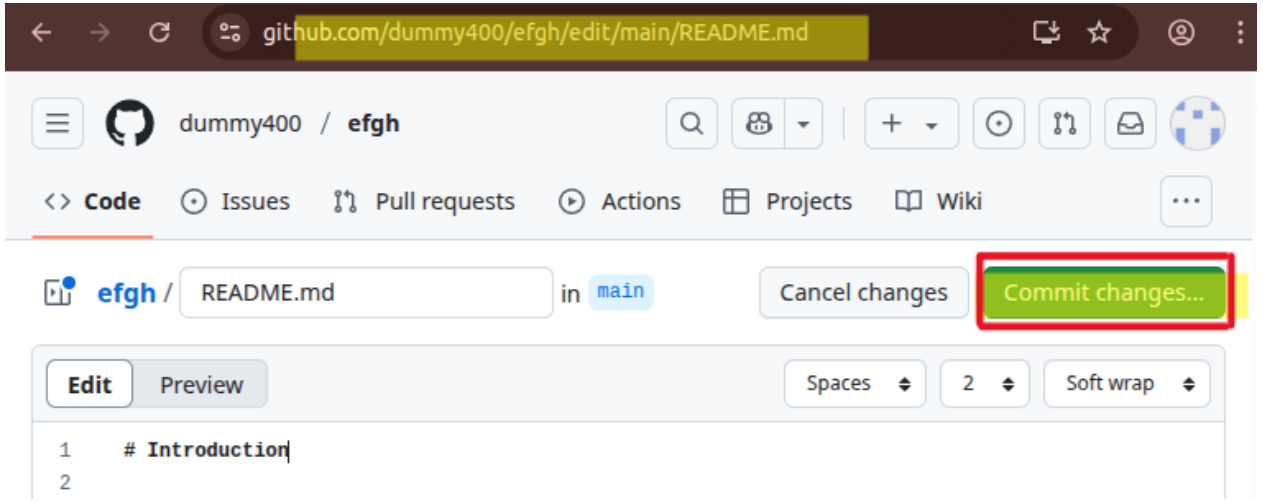
1 → Find likey bots from the top committers
2 → Distribution of commits among different categories
3 → Yearwise distribution of verified commits
4 → Unique committers with verified commits

Analysis. The other component of the tool analyses the data to find likely bots and distribution of commits across various categories and timeline.

Measurment Study

```
❯ python app.py analyze --task-name aiml1 --bots "gardener@tensorflow.org"
Repo: tensorflow/tensorflow
Total commits: 89277

 Top committers (Likely bots):

COMMITTER
gardener@tensorflow.org                77270
noreply@github.com                      1863
yong.tang.github@outlook.com             291
vikoth18@in.ibm.com                      278
advaitjain@users.noreply.github.com      270
Name: count, dtype: int64

Proportion of verified commits:

+------------+---------+---------------------------+
|            | Overall | Excluding bots and GH web |
+------------+---------+---------------------------+
|  Verified  |  2206   |            343            |
| Unverified |  87071  |           9801            |
+------------+---------+---------------------------+

Proportion of specific reasons for unverified commits:

+------------------+---------+---------------------------+
|                  | Overall | Excluding bots and GH web |
+------------------+---------+---------------------------+
|     unsigned     |  86763  |           9493            |
|      valid       |  2206   |            343            |
|   unknown_key    |   24    |            24             |
|     no_user      |    6    |             6             |
| unverified_email |   278   |            278            |
+------------------+---------+---------------------------+
```

Commits made via GitHub (GH) web are not considered in the analysis.

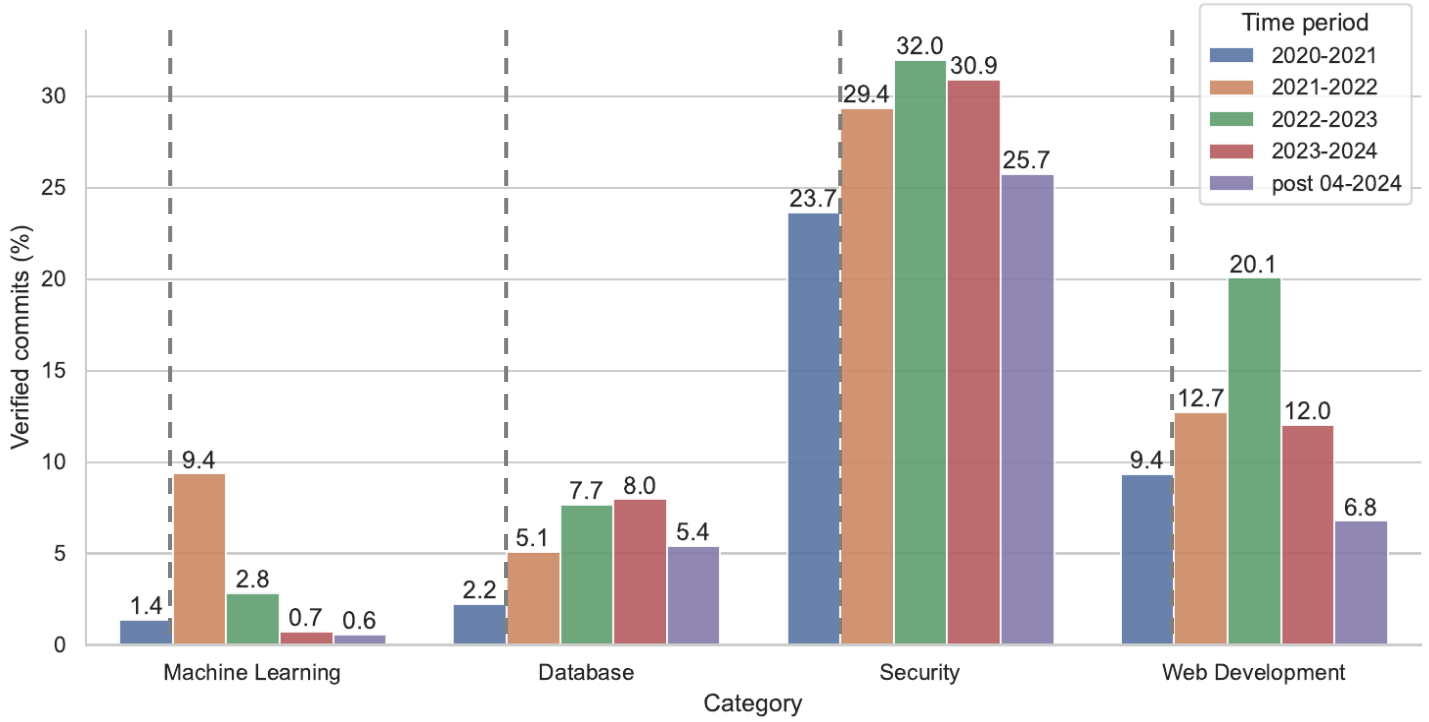


Commits made via GitHub (GH) web are automatically signed by GitHub.

- Commits made via GitHub web interface doesn't reflect commit signing practices and awareness of the users as the commits are automatically signed.
- We excluded commits made via GitHub web and bots in our analysis.

Measurment Study: Results

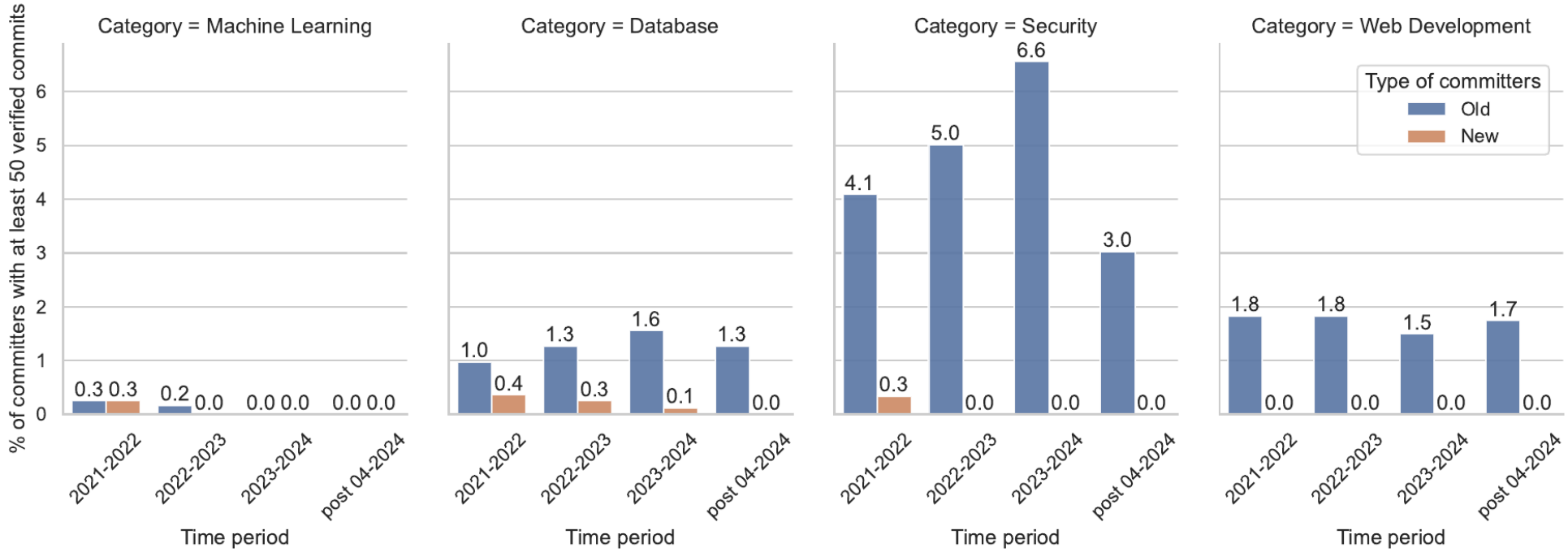
- The 289,308 commits (33.27%) of the total 869,437 were correctly signed.
- Excluding commits via GH web and bots, 38,586 (9.65%) of 399,520 were correctly signed.



Year-wise proportion of verified (valid or correctly signed) commits among different categories before and after GitHub announced vigilant mode and verification badges. This excludes commits made via bots and GitHub web. The dashed line marks the release of vigilant mode by GitHub (28 April 2021). Here, in the legends, the year starts and ends on the $28th$ day of April of the starting and ending year.

Measurment Study: Results

- What is the behaviour of new users in GitHub each year?



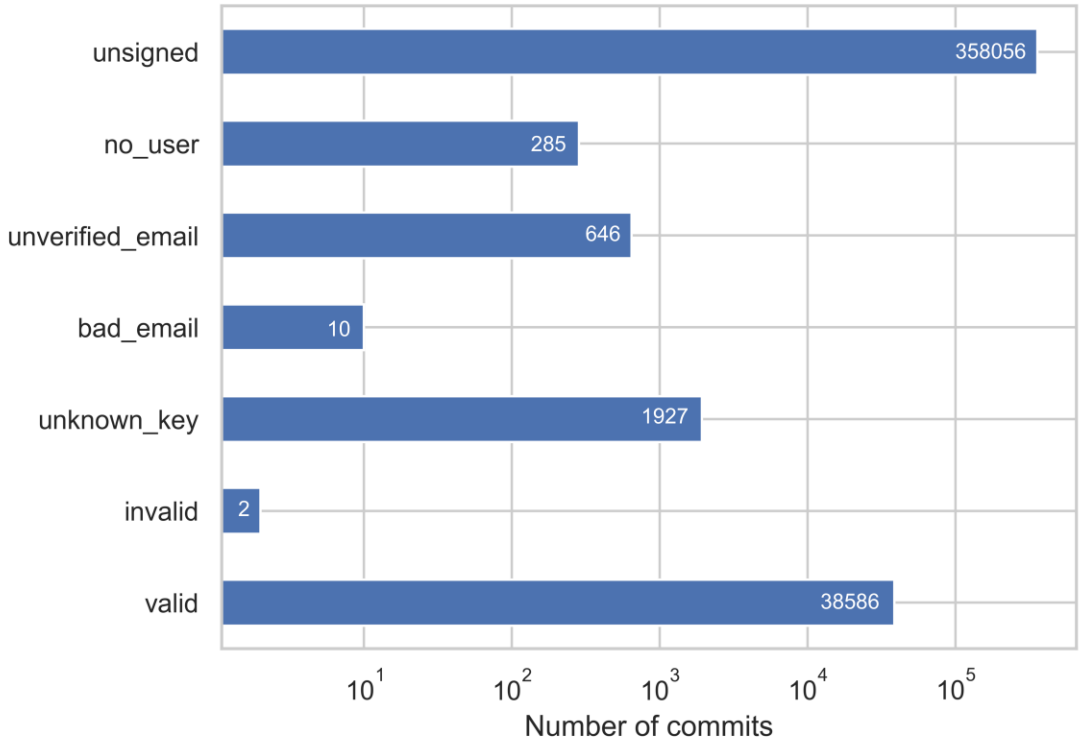
Proportion of old and new committers with at least 50 verified (correctly signed) commits each year. The rise and fall trend in the proportion of committers with verified commits could be the reason behind similar trend in previous figure. The drop in the proportion in the last interval could be due to the shorter range (4 months less than other intervals with 12 months).

Measurment Study: Possible reasons for low adaptation

- Though configuring commit signing is a one-time task, the procedure could be complex. Steps like generation of key pairs and their usage could be difficult to use by most users.
- Key management across multiple devices could be challenging.
- Enabling vigilant mode marks all older unsigned commits (possibly legit commits) as “Unverified”.
- Lack of support by Git clients for commit spoofing:
  - Git CLI: High learning curve for users with less experience with CLI.
  - GitHub Desktop:
    - No dedicated UI for commit signing.
    - No verification badges.
  - GitKraken:
    - User friendly UI for commit signing. No use of CLI needed.
    - Additional badges for signature verification. Works given the public key is imported.

Measurment Study: Results

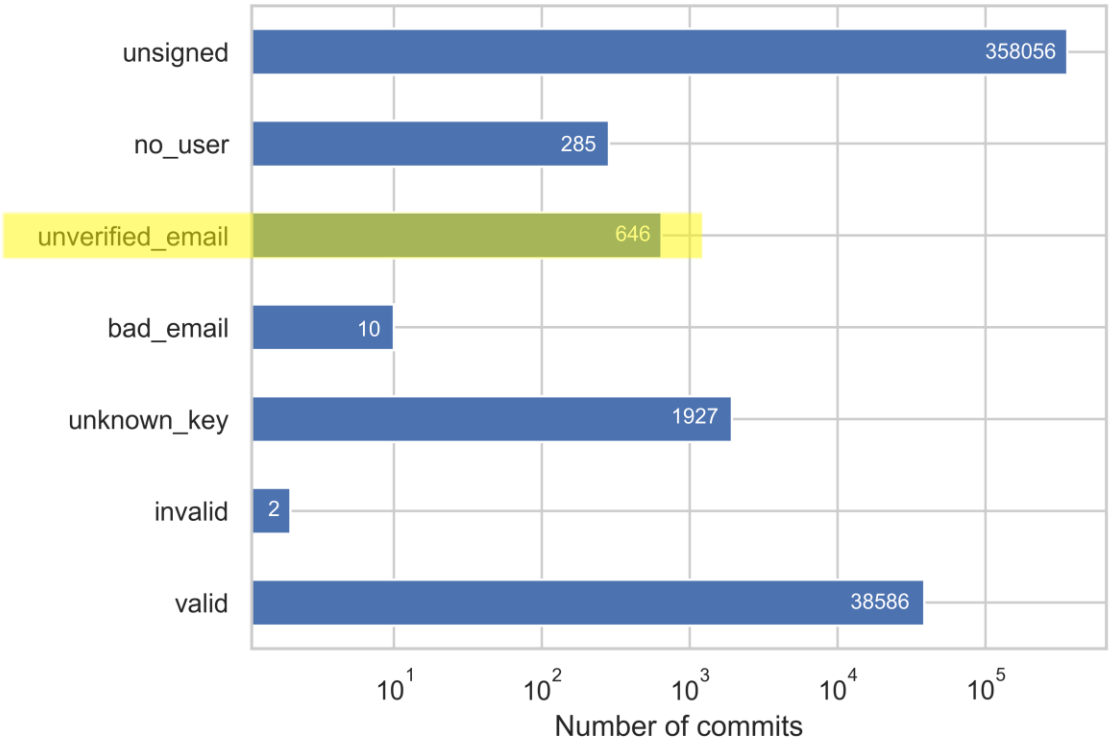
- Distribution of commits across categories.



- **Unsigned:** Unsigned commit
- **No user:** Email not associated with any GitHub accounts
- **Unverified email:** Email unverified in GitHub account
- **Bad email:** Email doesn't match with the one in the key
- **Invalid:** GitHub unable to verify signature
- **Valid:** Correctly signed commit

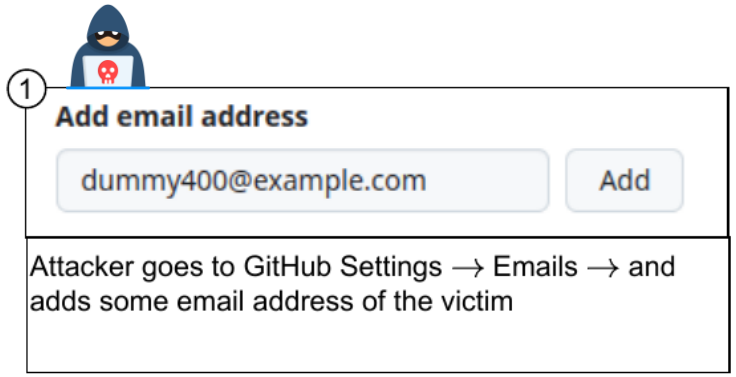
Measurment Study: Results

- Distribution of commits across categories.



- **Unsigned:** Unsigned commit
- **No user:** Email not associated with any GitHub accounts
- **Unverified email:** Email unverified in GitHub account
- **Bad email:** Email doesn't match with the one in the key
- **Invalid:** GitHub unable to verify signature
- **Valid:** Correctly signed commit

Email theft in GitHub



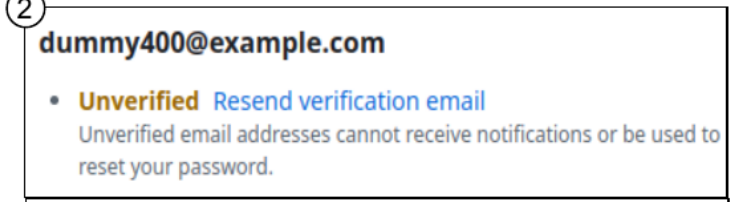
Attacker: dummy46 Victim: dummy400

Email Theft in GitHub

1

Attacker goes to GitHub Settings → Emails → and adds some email address of the victim

2



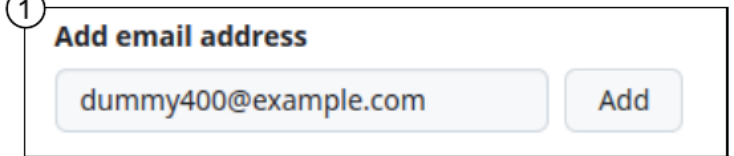
If the victim does not use the email with the GitHub profile, it gets added to attackers profile, but remains "Unverified"

Attacker: dummy46

Victim: dummy400

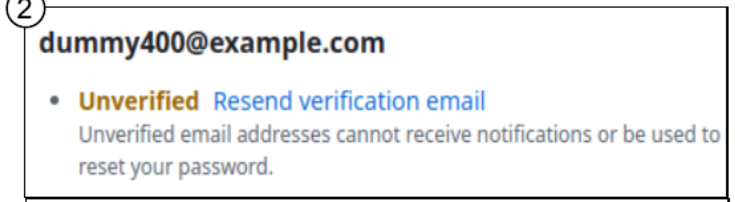
Email Theft in GitHub

1



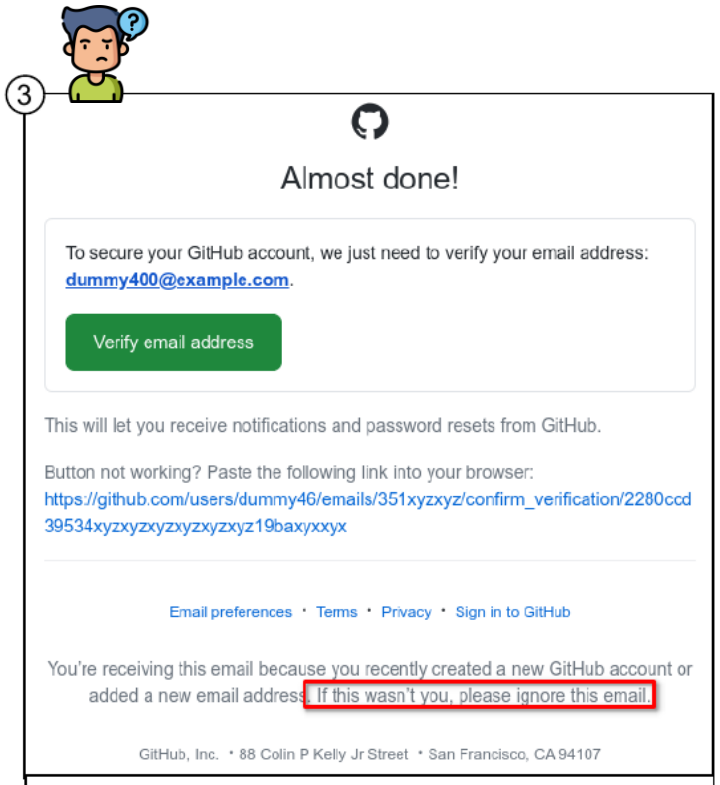
Attacker goes to GitHub Settings → Emails → and adds some email address of the victim

2



If the victim does not use the email with the GitHub profile, it gets added to attackers profile, but remains "Unverified"

3

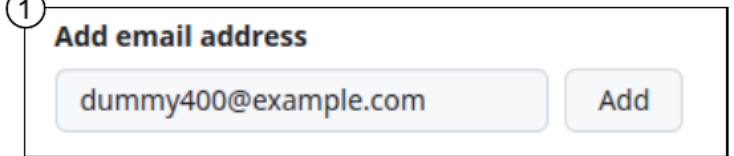


Verification mail similar to above goes to victims's mail. The victim will ignore this mail as it is instructed by the mail (highlighted in red) to ignore if it is not triggred by the victim

Attacker: dummy46 Victim: dummy400

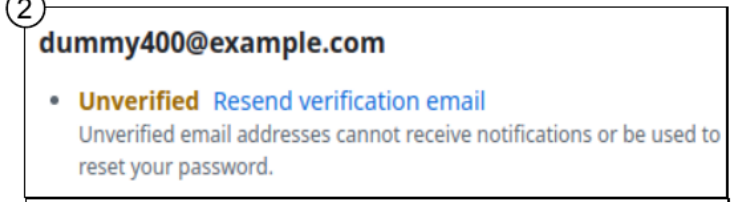
Email Theft in GitHub

1



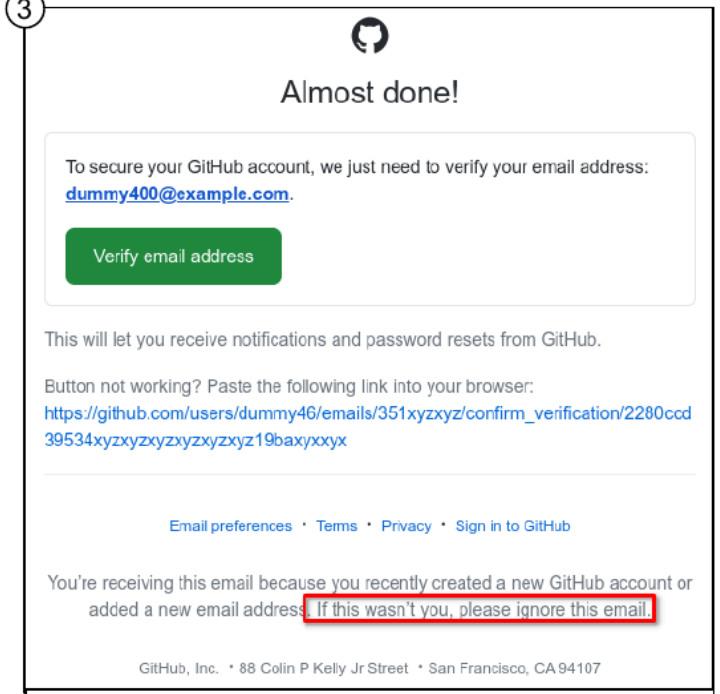
Attacker goes to GitHub Settings → Emails → and adds some email address of the victim

2



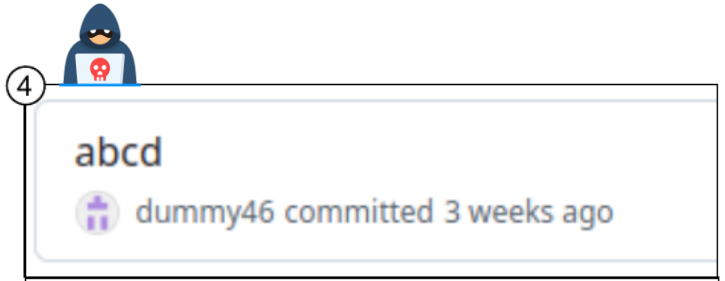
If the victim does not use the email with the GitHub profile, it gets added to attackers profile, but remains "Unverified"

3



Verification mail similar to above goes to victims's mail. The victim will ignore this mail as it is instructed by the mail (highlighted in red) to ignore if it is not triggred by the victim

4



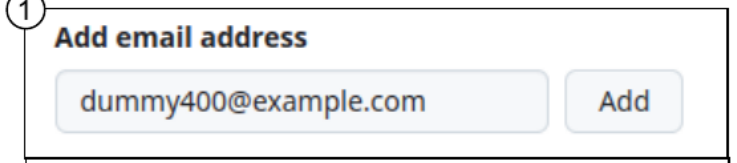
The attacker pushes a commit having victim's email in the commit meta data, and GitHub associates the commit with the attacker. But, since the username looks similar to victim's name having commit mail address same as victim's, arbitrary user may think that this account belongs to victim.

Attacker: dummy46

Victim: dummy400

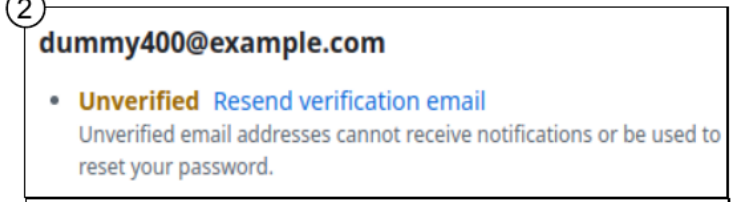
Email Theft in GitHub

1



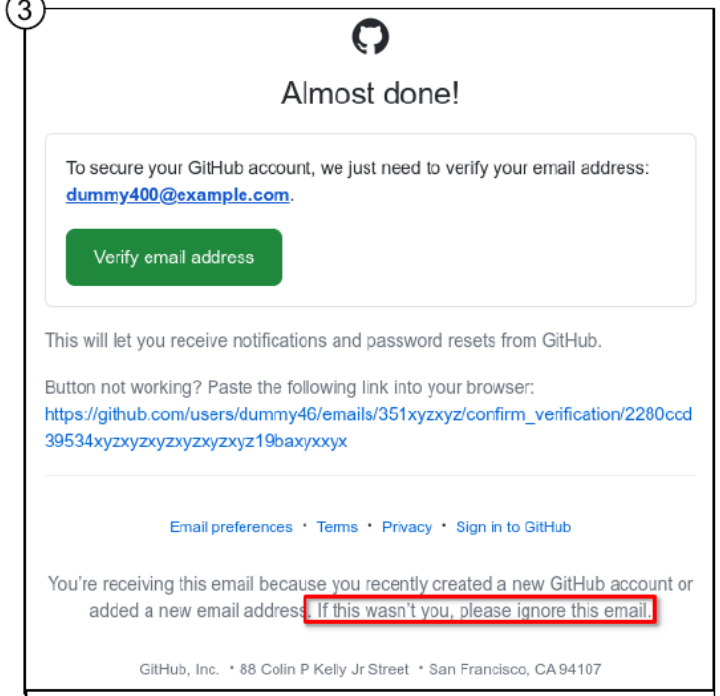
Attacker goes to GitHub Settings → Emails → and adds some email address of the victim

2



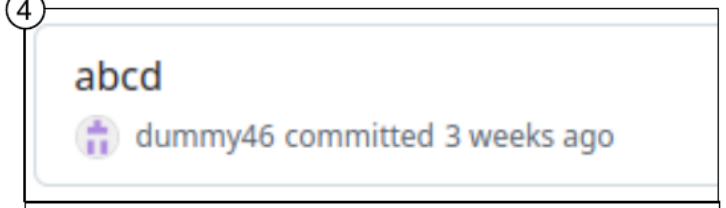
If the victim does not use the email with the GitHub profile, it gets added to attackers profile, but remains "Unverified"

3



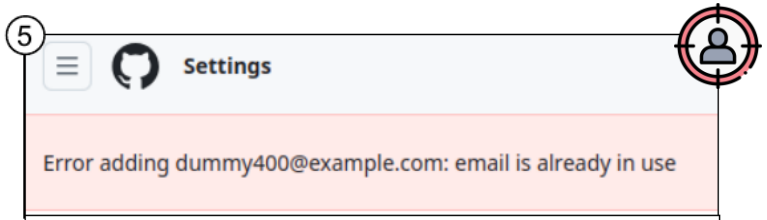
Verification mail similar to above goes to victims's mail. The victim will ignore this mail as it is instructed by the mail (highlighted in red) to ignore if it is not triggred by the victim

4



The attacker pushes a commit having victim's email in the commit meta data, and GitHub associates the commit with the attacker. But, since the username looks similar to victim's name having commit mail address same as victim's, arbitrary user may think that this account belongs to victim.

5



When the victim tries to add the email address to own GitHub account, GitHub fails to do so saying that it is already in use. Even though the attacker could not verify it via email verification as shown in step 2.

Email Theft: Email triggered when a user adds an email address to the GitHub account. If an attacker adds the email address to the account, the mail is sent to the owner. The owner has only two options: either to click on verify or ignore, as the mail clearly says (marked in red) to ignore if the owner does not trigger it. When ignored, the owner can never use the email address. **When the attacker use the unverified email address for a commit, the commit is successfully associated with the attacker's account. This may lead to typosquatting of user account leading to a situation having a kind of fake account without the user being aware.**

Email theft in GitHub

Mark (GitHub Support)
Apr 7, 2025, 3:02 AM UTC

Hi Anupam,

I do apologize for the delay in getting back to you, and I'm sorry to hear about the trouble here.

I'm afraid though, that the situation you describe here is an artifact of the underlying Git system used on GitHub, as that system attributes commits based on the authoring email address without any other verification. Given this, it would indeed be possible to add "another individual's email address" to any specific account, though such individuals can certainly reach out to GitHub Support to request assistance in reclaiming their address.

Please note that such an address cannot be verified without access to the email inbox, and so such commits would not be able to be signature verified, and so using this system can help with avoiding impersonation:

Managing commit signature verification

I will certainly pass on your feedback about including an option to contact GitHub Support in the verification mailers though, and if there is anything else you may like to add, then please let me know as well.

Regards,

Mark

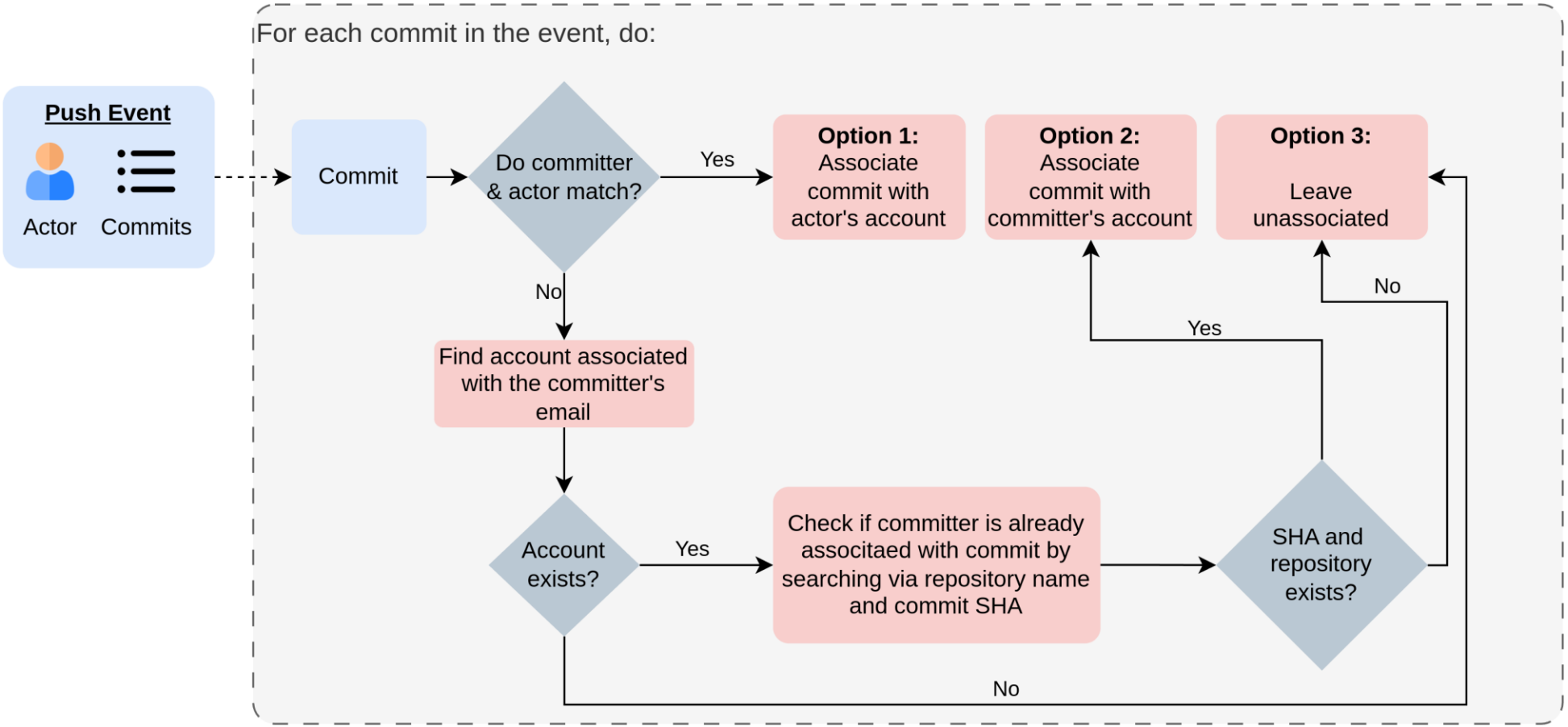
Response from GitHub: Currently, only solution is to contact GitHub support. But this is possible only when the user get to know about the theft. Other provided solution is commit signature verification. However, it cannot prevent this to happen. Moreover, we already saw a very low adaptation of commit signing.

Proposed Solution

- Leverage GitHub Events [4].
- For every activity performed in GitHub, an Event object is created.
- Out of 17 even types a local commit can be pushed to a repository in GitHub via:
  - Push Event
  - Pull Request Event
- Each events have the property named actor to identify the user that triggered the event.
- Idea: Match the identity in the commit and the identity of the actor triggering the event:
  - If matches, associate the commit with the account.
  - Else, leave the commit unassociated and mark it as a no_user commit.

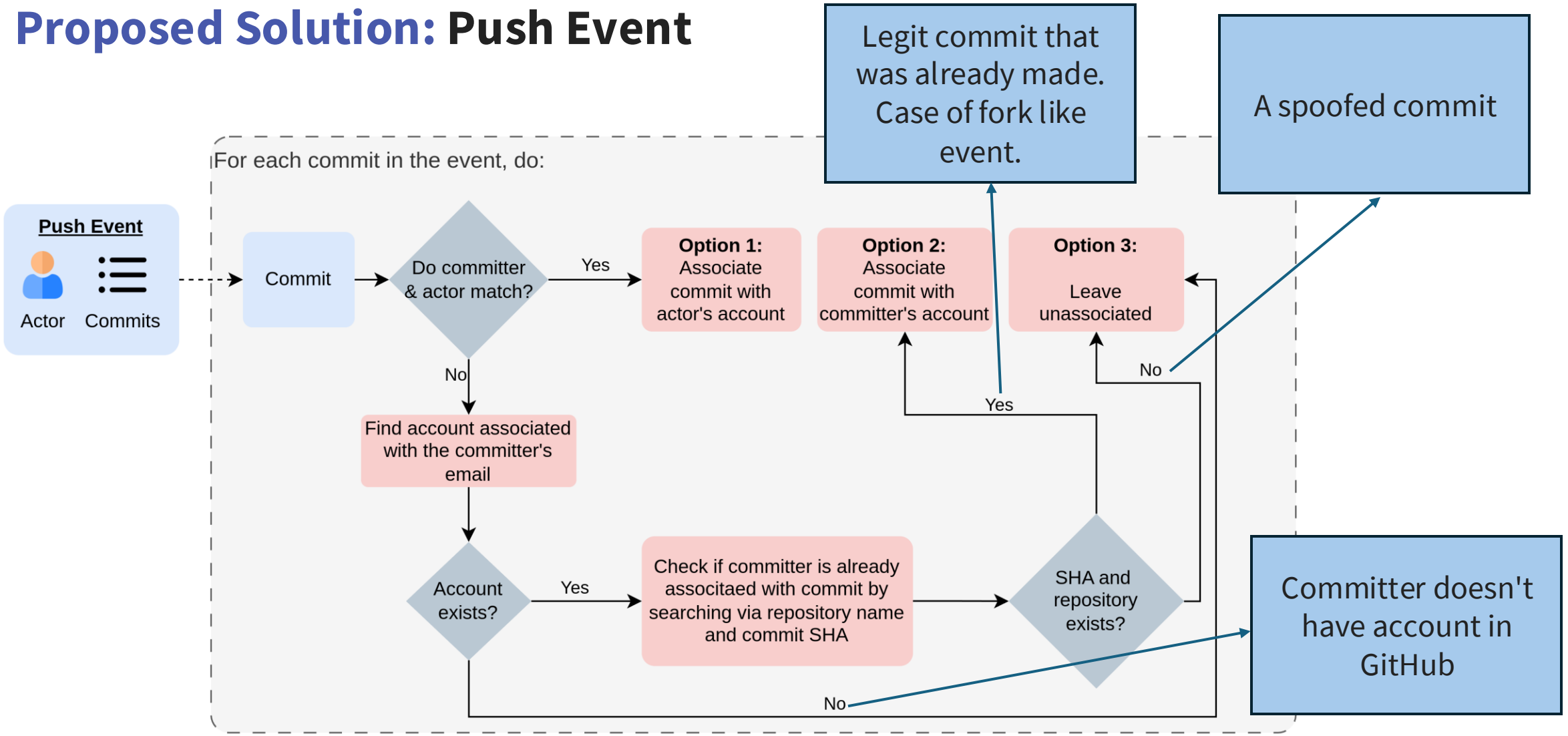
[4] GitHub. 2022. REST API endpoints for events — GitHub Docs. Retrieved October 10, 2024 from https://docs.github.com/en/rest/activity/events?apiVersion=2022-11-28#about-github-events

Proposed Solution: Push Event



Flow diagram for the proposed alternative for associating accounts with commit in GitHub. This process needs to be performed during the event.

Proposed Solution: Push Event



Flow diagram for the proposed alternative for associating accounts with commit in GitHub. This process needs to be performed during the event.

Proposed Solution

- Pull Request Event:
  - The action here can be anyone of the opened, edited, closed, reopened, assigned, unassigned, review_requested, review_request_removed, labeled, unlabeled, and synchronize.
  - Commits linked to the event are identified via the “commits_url" attribute [5].
  - If the identity in the linked commits and the identity of the actor of the pull request matches, the actor can be associated with commit.
- Email Theft:
  - Do not allow or associate commits with unverified email.
  - Provide an option in verification mail to report if the mail is not triggered by the user.
- Advantage:
  - Everything would be taken care inherently by GitHub. No need to make user aware of it.
  - No worries of older unsigned and legit commits marked as Unverified.

[5] GitHub. 2022. REST API endpoints for pull requests — GitHub Docs. Retrieved October 10, 2024 from https://docs.github.com/en/rest/pulls/pulls?apiVersion=2022-11-28#about-pull-requests

Conclusion

- The features provided by GitHub against commit spoofing is not well adapted by users.
- Users associated with Security repositories are more likely concerned of commit signing practices.
- New users committing repositories seems to be unaware of commit signing.
- Discovered email theft in GitHub due to allowance of unverified emails for committing.
- Provided recommendation for alternatives for association of commits and user accounts.

Thank You